

The Rise of Verbal Reinforcement Learning: A Survey

KSHITIJ TAYAL

ARUN SHARMA, University of Minnesota, Twin Cities, USA

GENTA INDRA WINATA

ANIRBAN DAS

SAMBIT SAHU

Large language models turned natural language from a task description into a learning signal: critiques, rubrics, reflections, and debates now steer agents where scalar rewards once did. We survey this practice as verbal reinforcement learning (VRL), organizing methods by the signal itself: what linguistic artifact is consumed, which decision-problem component it modifies, and when it takes effect. A formal framework with explicit inclusion criteria separates VRL from scalarized preference learning. We synthesize credit assignment for verbal signals, the verbal reward stack, language-space optimizers, embodied settings, evaluation, and feedback-channel safety, and distill decision rules for practitioners.

CCS Concepts: • **Computing methodologies** → **Reinforcement learning**; *Natural language processing*; *Intelligent agents*.

Additional Key Words and Phrases: verbal reinforcement learning, language feedback, large language model agents, credit assignment, reinforcement learning from feedback

ACM Reference Format:

Kshitij Tayal, Arun Sharma, Genta Indra Winata, Anirban Das, and Sambit Sahu. 2026. The Rise of Verbal Reinforcement Learning: A Survey. *ACM Comput. Surv.* 1, 1 (July 2026), 27 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

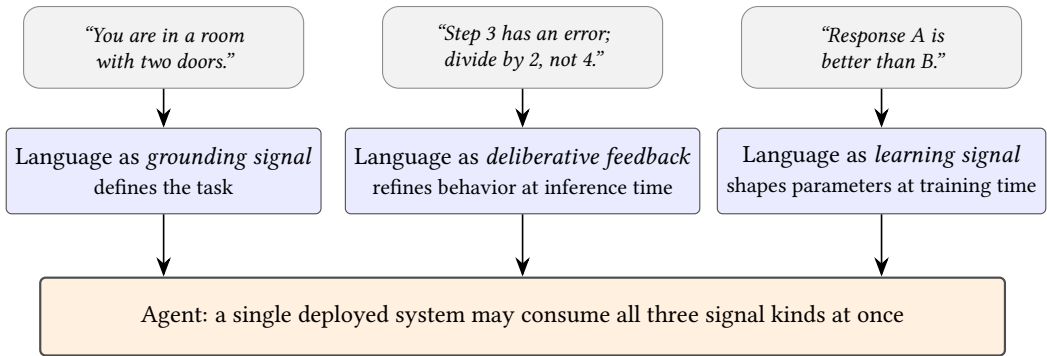


Fig. 1. Language plays three complementary roles as a signal in agentic systems: grounding the task and environment, scaffolding intermediate deliberation, and serving as a learning signal for training. The three roles classify signals, not system stages; a single deployed system may consume all three (§3).

Classical reinforcement learning assumes a specified environment, action space, and reward. Its landmark successes therefore depend on substantial domain engineering and on objectives

Authors' Contact Information: Kshitij Tayal; Arun Sharma, University of Minnesota, Twin Cities, Minneapolis, Minnesota, USA, sharm485@umn.edu; Genta Indra Winata; Anirban Das; Sambit Sahu.

2026. ACM 1557-7341/2026/7-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

whose optima match their designers' intent [56, 71, 81]. This specification burden is most acute for ambiguous, context-dependent tasks that resist scalar rewards [4].

Large language models open a second supervisory channel: language can express task intent, diagnose an output, preserve a lesson, or supply training feedback in a form an agent can consume directly [1, 7, 53]. The central question is no longer only how to derive a better scalar reward, but when retaining the structure of a critique, rubric, or instruction changes what can be learned.

We call the resulting family **verbal reinforcement learning (VRL)** and impose two membership criteria. First, feedback is expressed in natural language and its linguistic structure is functionally consumed, rather than collapsed to a scalar or preference bit before use. Second, it participates in an improvement loop through output revision, memory write, parameter update, or construction of a decision-problem component that later drives such an update. Both criteria are required. Plain instruction following fails the second. Vanilla RLHF [12, 58] and DPO [65] fail the first because the rationale is discarded before learning. Language-conditioned RL treats language as policy input while an ordinary scalar reward drives improvement. Reward-code generation remains inside the fence because a compiler consumes the language before the resulting reward drives optimization. Only feedback must be textual, so agents with visual observations remain in scope when their corrections are verbal (§10). Section 2 formalizes this fence.

We organize qualifying signals by when they take effect and what they modify. A *grounding signal* defines goals, states, actions, or reward structure at problem-definition time. *Deliberative feedback* revises context, output, or memory during inference without changing weights. A *learning signal* changes parameters or training data. These are roles of signals, not mutually exclusive system types or lifecycle stages: one agent can consume all three. Existing surveys usually organize by correction mechanism, agent component, or optimization family [60, 87, 114]; our signal-centric axis instead connects problem definition, inference, training, credit assignment, evaluation, and feedback-channel safety.

This article makes four contributions.

- (1) **Formalism and taxonomy.** A language-augmented decision framework locates linguistic artifacts at six entry points, while an explicit assignment rule resolves hybrid methods by the component modified when a signal takes effect (§2, §3).
- (2) **Mechanism synthesis.** We connect the three pillars to the verbal reward stack, credit assignment, and language-space optimization, distinguishing outcome, process, token, context, memory, and parameter updates (§7, §8, §9).
- (3) **Evaluation and safety.** We separate feedback quality from feedback utilization and analyze attacks and defenses at the feedback channel rather than treating the signal as trusted input (§11, §12).
- (4) **Practice and positioning.** We provide mechanism-selection rules, compare eighteen neighboring surveys and frameworks on fixed dimensions, and derive a focused research agenda (§13).

Sections 2 and 3 establish the formalism and taxonomy; Sections 4 through 6 develop the pillars. The remaining sections treat rewards, credit, optimization, embodiment, evaluation, safety, and practitioner decisions before Section 14 concludes.

2 A Formal Framework for Verbal Reinforcement Learning

The surveyed methods share a decision-theoretic skeleton but attach language to different components. We define that skeleton, identify where a linguistic artifact can enter it, and state the two criteria that separate verbal reinforcement learning (VRL) from neighboring paradigms.

2.1 The Language-Augmented Decision Process

Begin with a POMDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \Omega, T, R, O, \gamma)$ [33, 76]. Let $\mathcal{L}$ be the set of finite natural-language strings. A *linguistic artifact* $\ell \in \mathcal{L}$ is a string produced or consumed as a signal, such as a task description, critique, rationale, stored lesson, or reward-code fragment. The agent configuration $\kappa = (\theta, c, m)$ comprises model parameters θ , working context c , and persistent memory m ; policy π_κ maps interaction histories to action distributions.

The construct that distinguishes VRL from ordinary language-conditioned control is an *improvement operator*

$$\mathcal{I} : (\kappa, \ell) \mapsto \kappa', \quad (1)$$

which consumes an artifact and modifies the agent. Three realizations recur. An *output revision* changes c , for example by appending critique and regenerating [53]. A *memory write* changes m , preserving a lesson across episodes without changing weights [69, 121]. A *parameter update* changes θ and therefore future behavior. Operators may compose these effects, as when critique-generated revisions later become training data [5].

Language can enter the augmented process at six points.

- (i) *Goal specification.* A task description $g \in \mathcal{L}$ defines success when the true objective is latent.
- (ii) *State and observation.* Text environments, tool outputs, error traces, or verbalized sensor state provide the observation substrate.
- (iii) *Action prior.* Instructions or generated subgoals reshape the action distribution before learning [28].
- (iv) *Reward.* A generator may compile ℓ into $\hat{R} = C(\ell)$ at specification time [51], or a judge may evaluate trajectories at runtime [119].
- (v) *Transition prior.* Pretrained knowledge supplies predictions about action consequences, elicited in language [28].
- (vi) *Improvement.* Critiques, reflections, and rationales serve directly as the argument to $\mathcal{I}$.

The taxonomy maps signals acting on points (i) through (iv) at problem-definition time to grounding; output revisions and memory writes at inference time to deliberative feedback; and parameter updates to learning signals. The transition prior is a formal coordinate that the surveyed literature does not yet occupy with an explicit artifact. This POMDP is an organizing vocabulary, not a claim that every agent setting supplies a known transition kernel, reward function, Markov state, or stationary environment. In open-ended tool use and dialogue, those objects may be latent or undefined, so we do not import POMDP theorems merely because the notation is useful.

2.2 The Definitional Fence

A method belongs to VRL if and only if two conditions hold. First, feedback is expressed in natural language and its linguistic structure is functionally consumed rather than collapsed to a scalar or preference bit before use. Second, the feedback participates in an improvement loop through output revision, memory write, parameter update, or construction of a decision-problem component that later drives one of those updates.

Criterion (i) can be tested with a scalarization map $\sigma : \mathcal{L} \rightarrow \mathbb{R}^k$. If replacing ℓ by $\sigma(\ell)$ everywhere leaves the distribution over updated configurations κ' unchanged, the method does not functionally consume the language. Here k is fixed per judged unit: a preference bit, grade, rubric vector, or score per reasoning step is scalarized even when a trajectory contains many steps. Retaining text is not. Criterion (ii) requires that ℓ enter $\mathcal{I}$ directly or construct a component of $\mathcal{M}$ that subsequently drives improvement.

Because this test is counterfactual, we apply a dataflow inspection rule. Trace the artifact from producer to consumer and mark its *scalarization boundary*, where linguistic structure collapses into the numbers received by the learner. The criterion holds if a downstream consumer reads the artifact’s structure, such as a compiler parsing reward code [51], a reviser conditioning on critique [53], a retriever matching stored lessons [69], or a loss computed over feedback text. It fails when the sole downstream interface receives only a score. A rationale-annotated preference therefore qualifies only if the rationale enters revision or training; a step critique qualifies only if its text, rather than only its score, reaches the improvement machinery. The rule settles mechanism membership while leaving an empirical question: whether a consumer capable of reading linguistic structure actually exploits it.

Both criteria exclude familiar neighbors. Plain instruction following fails criterion (ii), since it conditions behavior without an improvement loop. Vanilla RLHF [58] and DPO [65] fail criterion (i) when the annotator’s rationale is discarded before the update. Language-conditioned RL [48] uses language as policy input while scalar reward drives improvement. Reward-code generation qualifies because a compiler consumes the full artifact before the resulting function trains a policy [51]. The feedback must be text, but observations need not be, which admits embodied agents receiving verbal corrections.

2.3 What Is Formally Guaranteed, and What Is Not

Formal results are recent and localized. Xu et al. [98] formalize learning from language feedback under latent rewards, define the transfer eluder dimension as a measure of feedback informativeness, and give the no-regret HELIX algorithm. Their separation result shows that rich language feedback can be exponentially more sample efficient than reward alone under their assumptions. He et al. [28] analyze hierarchical planning inside a POMDP: imitation of language subgoals can incur linear regret, while exploration restores sublinear regret when pretraining error is small. LLF-Bench operationalizes a related setting by replacing numeric rewards with varied natural-language instructions and feedback [10].

These results do not establish a general theory of VRL. No policy-improvement theorem covers $\mathcal{I}$, no convergence result covers language-space Bellman operators, and no bound covers self-generated, noisy, strategic, or adversarial feedback. Existing separations compare rich language with scalar reward but do not characterize intermediate formats such as step critiques or rationale-annotated preferences. Criterion (i) also lacks a measurable certificate of how much linguistic structure a learner uses. The framework therefore makes its claim boundary explicit: it supports classification and exposes the missing quantities, but it does not guarantee that critique, memory, or language-shaped reward improves a policy.

3 Taxonomy

The framework of §2 identifies where a linguistic artifact can enter an agent. We organize qualifying signals by *when* they take effect and *what* they modify, yielding the three pillars in Figure 2. The pillars classify signals, not systems: one agent may consume several roles at once. They are not lifecycle stages, and the definitional fence still applies at every leaf. In particular, grounding must feed an improvement loop, while scalar-only preference optimization remains outside VRL.

The leaves name one representative anchor per mechanism: SayCan [3], Closed-Loop State Feedback [6], Inner Monologue [30], Eureka [51], Self-Refine [53], ChatEval [9], Reflexion [69], Tree of Thoughts [102], FCP [49], STaR [105], DeepCritic [101], MALT [57], GenPRM [116], and DPO [65].

Grounding signals act at problem-definition time and specify goals, state descriptions, action interfaces, or rewards. They qualify only when the constructed component subsequently drives

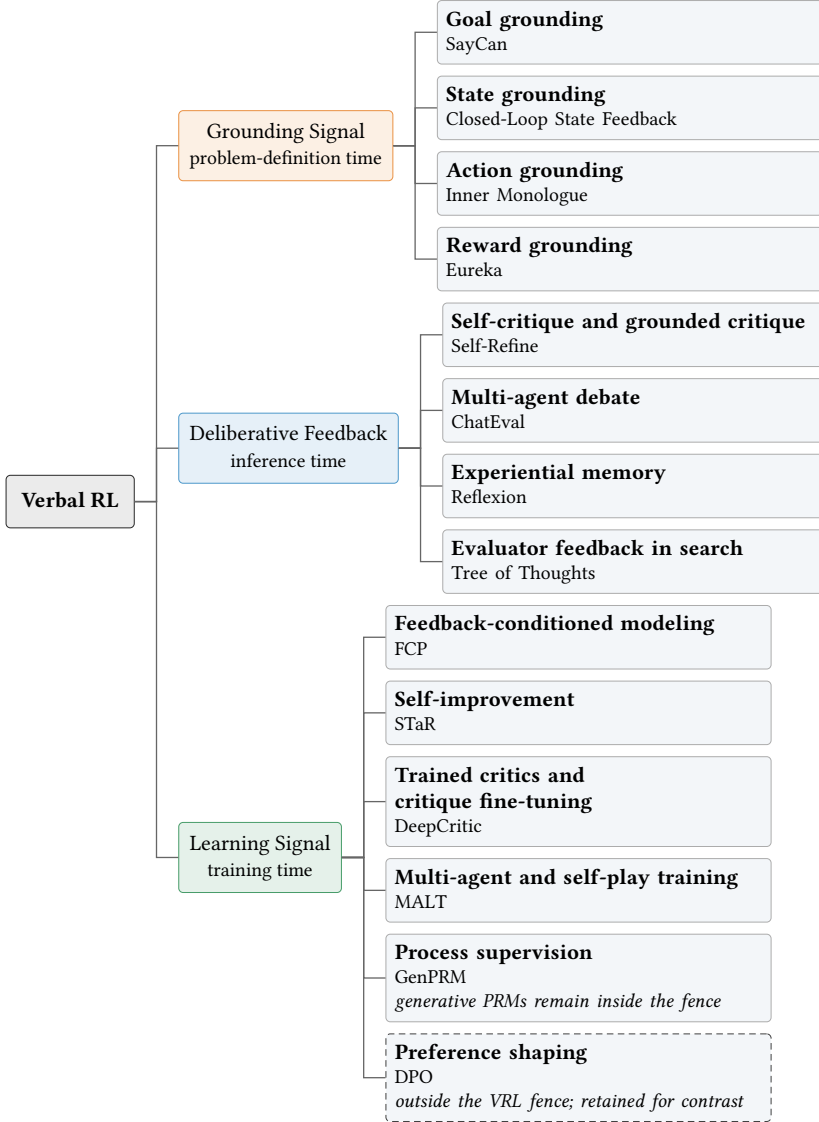


Fig. 2. The three-pillar taxonomy of VRL, organized by when a signal takes effect and what it modifies. Grounding examples qualify only when the constructed component drives an improvement loop. The dashed preference-shaping leaf is outside the VRL fence because the linguistic judgment is scalarized before learning.

revision or learning; a one-shot instruction to a frozen policy does not. Reward-code generation is the clearest case because language constructs the reward that trains the policy [51]. **Deliberative feedback** acts at inference time through critique, memory retrieval, debate, or search, changing context or output without updating parameters [53, 69]. **Learning signals** act at training time, where feedback changes parameters or training data and thus persists across future interactions [103, 105]. Persistence distinguishes the roles: grounding defines the task, deliberation changes an episode or memory state, and learning changes the policy.

Table 1. The assignment rule applied to recurring hybrid cases. Placement follows what the linguistic signal modifies at the moment it takes effect; secondary roles are tagged, not suppressed. P1 denotes grounding signal, P2 deliberative feedback, and P3 learning signal.

Hybrid case	Primary	Secondary role
Reward-code generation [51, 96]	P1	Constructed reward later trains a policy (P3)
Experiential memory [69, 77]	P2	Lessons persist across episodes, bordering long-term adaptation
Constitutional AI [5]	P3	Self-critique generates the revisions at inference time (P2)
Feedback-conditioned fine-tuning [49]	P3	Critique persists as conditioning context for generation (P2)

3.1 An Explicit Assignment Rule for Hybrid Methods

Hybrid methods require a reproducible placement procedure. We use one rule:

A method is placed by what its linguistic signal modifies at the moment the signal takes effect.

Table 1 applies the rule and tags secondary roles rather than suppressing them. Reward code is grounding when language constructs the task objective, even though that objective later trains a policy. Retrieved experience is deliberative because it changes inference context, even when memory persists across episodes. Constitutional critique and feedback-conditioned fine-tuning are learning signals when their durable effect is a parameter update. The same system may therefore appear under several pillars, but each artifact receives one primary placement at each point of use.

The temporal axis is preferable to source or modality because identical language can have different effects at different points of use. “The gripper pressure is too high” can constrain a task, revise a plan, or shape a compiled reward. Grouping by effect keeps together methods that share failure modes: grounding fidelity, deliberative feedback quality, or loss of information during training-time scalarization.

4 Pillar 1: Language as Grounding Signal

The first pillar contains signals that define the decision problem before behavior is evaluated. Language can specify the goal, abstract the state, constrain the action interface, or construct the reward. A method belongs here when the linguistic artifact modifies one of those task components and the resulting component drives an improvement loop. This placement distinguishes grounding from one-shot instruction following and from feedback that revises an existing trajectory. Across all four mechanisms, the binding constraint is not descriptive detail but whether language maps precisely enough to an executable component of the decision process [52, 76].

4.1 Goal Grounding

Goal grounding turns a verbal specification into target conditions that later optimization or revision can pursue. It must resolve objects and relations, filter plans by physical feasibility, and decompose an open-ended request into checkable subgoals. Systems such as SayCan combine linguistic relevance with affordance estimates, while language-specification methods train policies without a manually written reward [3, 39]. The central failure mode is compositional generalization: familiar primitives remain difficult when combined into a novel goal.

4.2 State Grounding

State grounding uses language as an observation abstraction. Text environments make this role explicit, while embodied agents obtain it by translating visual or physical state into descriptions. Transfer from verbal environments to embodied settings suggests that language can provide a modality-invariant representation, and closed-loop descriptions can support replanning [6, 70]. The relevant test is sufficiency, not fluency: a description fails when it omits history or state variables needed to distinguish actions with different values.

4.3 Action Grounding

Action grounding maps instructions to skills, tool calls, programs, or motor commands and returns execution outcomes to the planner. Inner Monologue illustrates the closed loop, using verbalized success and scene feedback to revise plans, while ProgPrompt constrains generation with program-like descriptions of available objects and actions [30, 72]. The principal design choice is granularity. A phrase may denote a long skill, a discrete subgoal, or a motor adjustment, and that choice fixes both the interface and the horizon over which credit must be assigned.

4.4 Reward Grounding: From Language to Reward Functions

Reward grounding compiles a verbal description of success into an executable objective. It is the key boundary case in the definitional fence: linguistic structure is consumed when the reward is constructed, even though the resulting function later emits scalars. Eureka searches over generated reward programs using textual reflection, and Text2Reward produces dense reward code that people can refine [51, 96]. When state is not directly available, learned vision-language judges can instead ground a text goal in observations [88]. These mechanisms fail differently from ordinary reward estimators. A miscompiled reward is a specification error that can be optimized confidently, so fidelity and hacking resistance must be tested before policy performance is trusted.

4.5 Synthesis: Shared Failure Modes and When to Ground

Three risks recur. The grounding gap separates a fluent description from an executable mapping. Compositionality failures appear when individually understood pieces interact. Fidelity failures arise when a valid task component does not mean what its author intended. Grounding is therefore the right first move when the task is not yet well posed: no programmatic objective exists, observations lack a useful encoding, or the action interface is unsettled. Once language has produced a testable problem, deliberative feedback can revise episodes and learning signals can reshape the policy. Detailed systems, quantitative comparisons, and secondary grounding variants are collected in the electronic supplement. The four roles also fail on different timescales. Goal and reward errors can contaminate an entire training run, state omissions recur at every observation, and action mismatches surface during execution. Validation should therefore occur at the interface each role constructs, before downstream optimization obscures the original grounding error.

5 Pillar 2: Language as Deliberative Feedback

Deliberative feedback refines behavior within an episode without updating model parameters, the regime closest to test-time compute scaling [73]. The four mechanisms differ mainly in where feedback originates: the acting model or an external tool, peer models, stored experience, or an evaluator inside search. A signal belongs here when its text is consumed in the current context; if the same artifact is converted into training data or weights, it becomes a learning signal. The organizing trade-off is among feedback quality, circularity, and inference cost.

5.1 Self-Critique and Externally Grounded Critique

Self-critique asks a model to diagnose and revise its own output. Self-Refine established the basic loop, but correction is limited when generator and critic share the same blind spots [34, 53]. External grounding changes the evidence available to the critic. Unit tests, execution traces, search results, and tool responses can expose errors that introspection cannot, as CRITIC demonstrates across reasoning and tool-use tasks [24]. This shift improves reliability but introduces latency, infrastructure cost, and a new interpretation problem: the tool may report the right symptom while the model infers the wrong cause. Grounded critique is therefore strongest where an inexpensive and auditable oracle exists.

5.2 Multi-Agent Debate

Debate distributes critique across model instances. Its benefit comes from genuine diversity rather than agent count alone: heterogeneous roles or models can reveal distinct errors, while identical debaters often converge to redundant consensus [18, 110]. Debate can also support scalable oversight when a weaker judge evaluates stronger participants, but it multiplies tokens and model calls. It is most defensible for problems that are hard relative to the base model and for which diverse reasoning paths are available; otherwise self-consistency or a single grounded critic is usually cheaper. Communication topology matters as much as participant count. Sparse exchange, selective retention of disagreements, and explicit confidence can reduce cost or prevent premature consensus. The relevant baseline is therefore not another debate protocol alone, but the strongest single-agent method at the same inference budget.

5.3 Experiential Memory

Experiential memory stores linguistic feedback for later episodes. The classification follows consumption: a retrieved lesson that conditions the current context remains deliberative even though the store persists. Reflexion anchors the raw-reflection regime, while ExpeL distills portable lessons from contrasts between successful and failed trajectories [69, 115]. What should persist depends on recurrence. Raw episodes suit closely repeated tasks, abstract principles transfer across related tasks, and executable procedures suit behavior that must be reproduced reliably. Store management becomes part of the mechanism as experience accumulates: selective writing, retrieval, pruning, and evidence-backed admission prevent stale or incorrect lessons from dominating. Memory differs from a language-space optimizer when the agent writes and reads its own store during episodes; an outer loop that edits a prompt or playbook against a task metric instead treats text as a system parameter. Because retrieval changes the effective context, memory evaluation must test more than recall. A useful store should improve behavior on later tasks, reject irrelevant lessons, preserve raw evidence when summaries are uncertain, and forget entries whose utility has decayed.

5.4 Evaluator Feedback in Search

Search methods branch over candidate trajectories and use linguistic evaluation to choose which nodes to expand or abandon. Tree of Thoughts and Reasoning via Planning are representative designs [27, 102]. Their advantage is early rejection of unproductive paths; their cost is repeated evaluation across many states. Sampling or voting without a verbal evaluator is outside the survey's fence. Search is preferable when partial trajectories can be judged before completion and when that early pruning saves more computation than the evaluator consumes. Otherwise, evaluator noise can discard the correct branch while adding cost.

Synthesis: when this pillar wins: Feedback quality is the first decision variable. Self-critique is cheap but circular, debate helps only when participants contribute distinct evidence, tools are

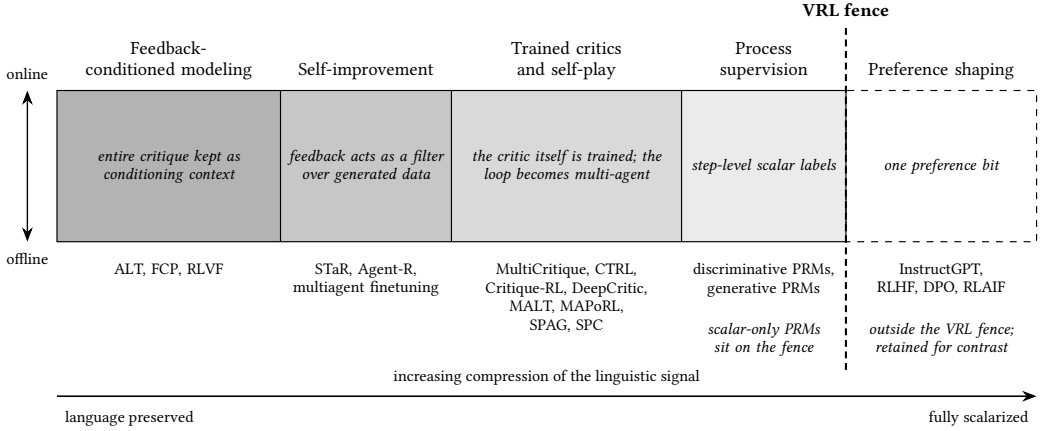


Fig. 3. Training-time methods ordered by how much linguistic structure survives into the parameter update. The dashed fence places scalar-only preference optimization outside VRL and scalar-only process supervision on its boundary. Online and offline collection cut across every band.

reliable only where a verifier exists, and memory can entrench early mistakes. Compute is the second variable: debate and search spend additional calls within an episode, while memory amortizes prior deliberation across episodes. Deliberative feedback is therefore the preferred mechanism when weights are inaccessible, tasks are difficult enough to justify iteration, or recurring tasks let stored lessons pay for themselves. When training access and a stable task distribution are available, the same critiques are often better internalized as learning signals. Extended system catalogs, benchmark results, memory-management variants, and the full mechanism diagram appear in the electronic supplement.

6 Pillar 3: Language as Learning Signal

The third pillar covers signals that persist through parameter updates. We organize its methods by how much linguistic structure survives into training. Feedback-conditioned modeling retains an entire critique; self-improvement uses feedback to filter generated data; trained critics and multi-agent learning optimize the producer of feedback; process supervision localizes judgments to steps; and preference shaping compresses a judgment to one comparative bit. Richer signals preserve actionable information but require a learner capable of consuming text, while scalar compression enables mature optimization machinery at the cost of expressiveness. An orthogonal online-offline axis records whether feedback is collected from the current policy or inherited from a fixed corpus. Figure 3 summarizes both distinctions.

6.1 Feedback-Conditioned Modeling

At the least-compressed end, the learner receives the critique together with the response it should produce. ALT shows why retaining the critique matters, and the feedback-conditional policy formalizes training as maximum likelihood over response-feedback pairs followed by optional online bootstrapping [46, 49]. This family preserves corrective detail and supports targeted scope, but it also risks teaching indiscriminate compliance with inaccurate feedback.

6.2 Self-Improvement

Self-improvement uses language as a filter over model-generated trajectories. STaR retains reasoning paths that reach correct answers, while Constitutional AI filters revisions against written principles [5, 105]. The approach scales cheaply because training consumes ordinary supervised examples. Its bottleneck is filter quality: when the same model generates and judges candidates, systematic errors can compound across rounds.

6.3 Trained Critics and Critique-Based Fine-Tuning

A trained critic makes feedback production an explicit capability. CTRL optimizes a critic for the downstream correction it induces, while DeepCritic combines deliberate critique with reinforcement learning over step labels [97, 101]. The design is useful when a frozen general-purpose judge limits improvement, but critic evaluation must separate helpfulness from discriminability. A critic that produces persuasive but poorly localized feedback can improve revisions while remaining an unreliable training signal.

6.4 Multi-Agent and Self-Play Training

Jointly training feedback producers and consumers turns the loop into a multi-agent learning problem. MALT propagates outcome credit through generator, verifier, and refiner roles, while MAPoRL co-trains discussion participants for corrective and persuasive contributions [57, 61]. Self-play can reduce external annotation, but co-adaptation creates its own instability and may reward persuasion rather than truth. The transcript's destination determines placement: context-only debate is deliberative, while transcripts or rewards that alter weights belong here.

6.5 Process Supervision

Process supervision attaches judgments to intermediate reasoning steps. Scalar click labels alone fail the linguistic-structure criterion, even when one score is provided per step. Generative process reward models remain inside the fence because language is consumed during reward computation, and free-form step critiques qualify when their content determines where credit lands [41, 116]. Finer granularity improves localization but raises annotation and verification cost, especially outside mathematics and code.

6.6 Preference Shaping: The Scalar Endpoint

Preference shaping collapses a comparative judgment to one bit. InstructGPT established the reward-model plus policy-optimization recipe, and DPO trains directly from preference pairs [58, 65]. These methods are retained as the compression endpoint, not as VRL proper, because the learner never consumes the rationale. Online collection can follow new policy failures; offline objectives inherit the coverage and biases of a fixed corpus.

Synthesis: compression versus optimizability: The spectrum is a conditional design rule. Preserve critiques when feedback is trustworthy and a consumer can use their structure; filter generated data when a cheap verifier exists; train critics when feedback production is the bottleneck; use process signals when localization justifies their cost; and scalarize only when throughput matters more than explanatory content. Rich feedback can improve sample efficiency, but stale offline feedback and self-generated judgments introduce coupled bias. The electronic supplement records the secondary systems, algorithmic variants, and benchmark evidence behind each band.

Table 2. The verbal reward stack, ordered by scalarization boundary and signal granularity.

Rung	Reward computation	Scalarization boundary	Granularity	Representative work
Verifiable outcome	Programmatic check against ground truth	At the environment; no linguistic structure in the reward	Episode	[26, 38]
Rubric reward	Judge checks instance-specific criteria; item scores pooled	At aggregation of criterion verdicts	Response	[25, 82]
Generative reward model	Judge writes principles and critique, then a verdict	At verdict emission; critique retained as an audit trail	Response	[45, 111]
Process reward	Step-level verification, increasingly verbalized	Per step, or per token at the limit	Step or token	[35, 86]

7 The Verbal Reward Stack

Training-time VRL depends on how an evaluation becomes the reward consumed by an optimizer. Four constructions form a progression: verifiable outcome rewards, rubric rewards, generative reward models, and process reward models (PRMs). The rungs are ordered by two properties: where linguistic structure collapses into a scalar, which we call the *scalarization boundary*, and the granularity at which the resulting signal attaches to a trajectory. Moving upward retains language later and localizes credit more finely, but increases verification cost. Table 2 summarizes the stack.

7.1 Verifiable Outcome Rewards

Reinforcement learning with verifiable rewards uses programmatic correctness checks. Tulu 3 established the open post-training recipe, and DeepSeek-R1 showed that outcome-only reinforcement can elicit verification behavior without labeled reasoning traces [26, 38]. This is the degenerate rung: scalarization occurs in the environment and no language appears in the reward. Its strengths are exactness and low cost; its limits are sparse feedback, narrow domain coverage, and evaluation sensitivity to compute budgets. Outcome verification is therefore the default only when correctness is both mechanically testable and difficult for the policy to game. Otherwise it supplies a reliable terminal label but little guidance about how a failed trajectory should change.

7.2 Rubric Rewards

Rubrics extend verification to open-ended tasks by decomposing success into instance-specific criteria. Rubrics as Rewards demonstrates that structured criteria can outperform direct holistic judging, while checklist rewards apply the same idea to instruction following [25, 82]. Static banks amortize authoring cost, dynamic criteria adapt to policy drift, and selective checking saves judge calls. In every case, scalarization occurs when criterion verdicts are aggregated, so any omitted requirement becomes invisible to the optimizer. A rubric should therefore be treated as a versioned specification, with coverage audits and updates tied to new policy failures rather than as a fixed scoring prompt.

7.3 Generative Reward Models

Generative reward models verbalize the evaluation before emitting a verdict. Generative verifiers write a rationale and benefit from sampling or voting, while DeepSeek-GRM generates principles

and critiques before scoring [45, 111]. Their text can improve judgment and provide an audit trail, but the optimizer still receives a scalar unless the critique is separately routed into revision or training. Correct verdicts do not guarantee sound explanations, so critique quality must be supervised or evaluated independently. Their extra inference cost is justified when hard cases require adaptive reasoning or when the retained critique supports auditing, downstream revision, or later credit assignment.

7.4 Process Reward Models

PRMs move evaluation inside the trajectory. Math-Shepherd automated step labeling, and ThinkPRM generates a verification rationale for each step [35, 86]. Fine granularity can localize the first error and improve credit assignment, but noisy step labels and weak transfer beyond curated reasoning tasks remain central limitations. Generative PRMs connect this rung to verbal feedback because their linguistic reasoning participates in computing each local score. They are most useful when intermediate steps have stable semantics and can be checked independently; otherwise the apparent precision of a per-step score can exceed the reliability of its annotation.

7.5 The Scalarization Boundary

Every rung ultimately sends numbers to the optimizer. Verifiers scalarize at the environment, rubrics after criterion aggregation, generative judges at verdict emission, and PRMs per step. What disappears is the repair information carried by language: which failure occurred, where it occurred, and what should change. Finer granularity narrows that loss but does not eliminate it; language-space optimizers do so by applying the critique as text.

The boundary is also where exploitation concentrates. A policy can target whatever a verifier, rubric, or judge fails to communicate through its scalar output. Outcome checks invite shortcut reasoning, static rubrics leave exploitable omissions, generative judges can pair correct verdicts with unsound critiques, and noisy step labels corrupt PRM training. The practitioner rule is therefore to choose the lowest-cost rung that captures the distinctions the task requires, then test the scalarization boundary for omitted criteria, judge error, and hacking. Extended reward-model variants, quantitative results, and defense evidence appear in the electronic supplement.

8 Credit Assignment and Optimization with Verbal Signals

Credit assignment asks which parts of an agent's behavior deserve blame or praise for a delayed outcome. Language makes this problem harder because an LLM action may span thousands of tokens and several turns, yet it also offers a distinctive solution: a critique can name the faulty step and the needed repair, information that no scalar carries. We organize the literature by the granularity at which credit lands (outcome, step, turn, token, or memory operation) and by its carrier (redistributed scalar, model-internal attribution, or critique text). These choices determine which optimizer can consume the signal and which assumptions must hold.

8.1 Outcome, Process, and Reflective Credit

Three levels recur. *Outcome credit* scores only the final trajectory. It is cheap and exact for verifiable tasks, which makes it suitable for group-comparison methods, but broadcasting one scalar over every token is maximally coarse [26]. *Process credit* judges intermediate steps; process supervision and process reward models make a delayed outcome locally actionable [41, 80]. *Reflective credit* uses the content of an evaluation ℓ , rather than only its scalarization $\sigma(\ell)$, to determine where and how an update lands. Stored reflections steer later attempts, critiques can be aligned to the spans they indict, and feedback-conditioned policies can retain text as a conditioning variable instead

Table 3. Families of credit-assignment mechanisms in VRL, ordered by how much linguistic content the credit carrier consumes.

Family	Credit carrier	Granularity	Representative work
Temporal abstraction	scalar reward over macro actions or turns	span, turn	[8, 89, 123]
Reward redistribution	holistic scalar re-allocated after the fact	token or time step	[40, 64]
Model-internal attribution	explainability scores or attention flow	token	[16, 37]
Information-theoretic	belief update about the final answer	turn	[83]
Generative process critique	step-wise critique converted to credit	step, token	[44, 84, 95, 122]
LLM-as-annotator	subgoal and action judgments by an external LLM	action, subgoal	[42, 63, 120]
Memory-conditioned	credit assigned to memory operations	memory write	[99, 121]

of compressing it [49, 69, 84]. Outcome credit ranks trajectories, process credit ranks steps, and reflective credit can state what to change.

8.2 Mechanisms and Selection Rules

Four mechanisms push sparse outcomes toward the responsible behavior. Temporal abstraction shortens the horizon by treating token spans or turns as macro actions [8, 123]. Reward redistribution preserves a trusted sequence-level scorer but reallocates its value across tokens or time steps [40, 64]. Model-internal attribution derives token importance from additive explanations or information flow, which makes the shaping signal inspectable but ties it to the reward model's internals [16, 37]. Critique-derived methods generate stepwise diagnoses and convert them into token or step credit, gaining semantic localization without a separately labeled dense reward model [44, 95, 122].

The choice follows the artifacts available before training. Temporal abstraction is the cheapest first move when a trusted outcome reward exists. Redistribution is appropriate when the reward model is the only trusted component; attribution is preferable when the credit trace must be inspected. Critique-derived credit is the only family that can both localize and name an error, but it is justified only when critic quality can be audited. LLM annotation extends the same idea to non-language policies by judging subgoals, transitions, or team contributions, moving the principal failure risk into the annotator [42, 63, 120].

8.3 Long Horizons and Memory

Multi-turn interaction introduces two nested questions: which turn helped, and which tokens within it. Hierarchical learners combine turn-level values with within-turn policy gradients; explicit turn rewards are lighter when intermediate outcomes are checkable; information-gain rewards use the policy's belief update when no external reward model is available [83, 89, 123]. Persistent memory adds a state-conditioned credit problem. Because memory writes change the effective environment seen by later rollouts, group-relative comparisons become biased unless memory operations are compared from a shared intermediate state [99, 121]. Turn-level rewards are therefore an optional refinement, but memory-conditioned credit is mandatory once the agent writes persistent state.

8.4 Optimizer Pairing

Feedback form constrains the optimizer. Executable rewards pair naturally with online policy gradients; pairwise judgments with offline preference objectives; filtered or feedback-conditioned text with supervised fine-tuning; and step-level judgments with process reward models [41, 49, 65, 66]. Group comparison adds an assumption that rollouts share an environment and reward scale, which turn structure and memory writes can violate [26, 89, 99]. Language-mediated states also risk information loss, and language-generated actions alter exploration. Planner-imitated subgoals, for example, require explicit exploration to avoid linear regret [28]. The practitioner should therefore choose the feedback carrier first and treat the optimizer as a downstream consequence.

8.5 The State of Theory

General theory remains thin. The main formal anchor is learning from language feedback through transfer eluder dimension [98]. Partial results cover regret for language-mediated hierarchies, optimality preservation under feature-additive shaping, redundancy reduction for latent rewards, and the derivation of implicit step rewards [28, 37, 44, 64]. Missing are conditions under which critique-derived token credit preserves the optimal outcome policy and bounds on how verbal localization errors compound through training. This gap is central to the research agenda in §13.

9 Language-Space Optimizers

The improvement operator of §2 can revise one output, write persistent memory, or update model parameters. Language-space optimization occupies the middle ground: an outer optimizer treats a prompt, context, playbook, or scaffold as an editable system parameter and improves it against a task metric. It is therefore not a fourth pillar. The optimized text persists across episodes like a learning artifact, but a frozen model consumes it through the context window. This semi-parametric view also clarifies the analogy to numerical optimization: critique supplies directional information, text is the parameter, an LLM edit is the update, and execution traces route credit through compound systems [11, 93].

9.1 Mechanism Families

Four families differ in the information supplied to the update. *Score-driven search* proposes candidates and retains those with higher task scores; OPRO places previous candidates and scores in context so the optimizer can exploit its search history [100]. *Textual-gradient methods* replace a scalar score with a diagnosis and propagate it to the responsible prompt or program component; TextGrad is the canonical realization, while trace-based methods extend the idea to compound graphs [104]. *Reflective and evolutionary methods* maintain diverse candidates, diagnose failures, and select complementary improvements; GEPA shows that richer diagnoses can achieve stronger sample efficiency than policy-gradient updates when rollouts are scarce [2]. *Hybrid methods* alternate text edits with weight updates, using prompt evolution to escape failures and then distilling the gain into the model; P²O exemplifies this division of labor [47]. Detailed systems and quantitative comparisons appear in the electronic supplement.

The families also assign credit at different resolutions. Score-only search credits a whole candidate, which is simple but expensive when pipelines contain many coupled prompts. Textual gradients use execution traces to localize blame to a component. Reflective populations trade precise attribution for diversity, retaining several explanations or solutions when no single update dominates every objective. Hybrids add a second choice: whether a discovered improvement should remain explicit in context or become procedural knowledge in weights. This credit unit, candidate, component,

population, or parameter update, is often more consequential than the particular LLM used as optimizer.

9.2 Selection Rules and Failure Modes

Language-space optimization is preferred when weights or gradients are inaccessible, supervision is scarce enough that explanatory feedback must extract more information per rollout, or the learned artifact must remain inspectable and editable. Trace-aware textual gradients are especially useful when a compound pipeline needs component-level credit. Population methods help when several objectives or qualitatively different solutions must be retained. Weight updates remain preferable when abundant data amortizes training, the target knowledge is procedural, or deployment cannot afford a growing context. Hybrid methods fit the intermediate case, where an interpretable prompt discovers a behavior that must later be internalized.

The same properties create predictable failures. A reflective optimizer can misdiagnose the cause of an error and move in the wrong direction [50]; wholesale context rewriting can collapse useful detail [112]; multi-objective critiques can dilute task focus [14]; and poisoned feedback directly poisons the update channel. Two contrast cases mark the definitional fence: text optimized by actual numeric gradients or by selection signals with no linguistic meaning is prompt optimization, but not verbal reinforcement learning [15, 85]. The verbal channel primarily buys sample efficiency, auditability, and an interface for structured credit, not a guarantee of higher attainable performance.

10 Language Feedback in Embodied and Multimodal Settings

Embodied systems receive images, video, and proprioception and emit motor actions, so language may appear only in the feedback channel. Here *verbal* denotes the structure of the consumed signal, not its carrier: typed text, transcribed speech, or a vision-language model’s explanation qualifies when compositional language changes behavior. A score derived from the same observation does not. Language-conditioned control also remains outside scope when the instruction specifies the task but does not evaluate prior behavior.

10.1 Four Embodied Roles

Corrections map a phrase to a policy change during execution. LILAC grounds real-time utterances in a low-dimensional control space, illustrating the central challenge: the system must identify both what failed and which motor degree of freedom should change [13]. Corrections can persist as memory or training data, but their taxonomy assignment follows where the language is consumed.

Language-to-reward uses linguistic feedback to define or revise an objective before policy optimization. Eureka verbalizes training statistics and mutates reward code, making reward design itself a language-space optimization loop [51]. This family fits simulators that expose state and optimization, but inherits reward misspecification and hacking risk.

Vision-language rewards and critics judge behavior when only pixels are available. RL-VLM-F learns rewards from vision-language preference labels over observations [88]; process variants provide denser temporal credit. Their main risk is partial observability: a judge may recognize objects while missing sequence, motion, or causal progress.

Failure reasoning produces an explanation that a planner or controller can use for recovery. AHA detects failures in free-form language and improves downstream planning, verification, and reward refinement when inserted into the loop [19]. Unlike a scalar reward, the explanation can identify the failed subtask and suggest a repair, but it still requires grounding back into action.

These roles can coexist in one system without becoming interchangeable. A spoken correction first steers the current action, then becomes memory if stored, and finally becomes a learning signal if logged for retraining. Likewise, a vision-language explanation remains deliberative while

it guides replanning, but becomes objective-side supervision when converted into reward code. Classification follows the consumed representation and update locus at each use, not the sensor or model that produced it.

10.2 Selection Rule

Choose human corrections when a person can intervene online; generated rewards when a simulator exposes state; vision-language rewards when observation is pixel-only and scalable supervision is required; and failure critics when autonomous recovery matters. These roles enter different parts of the decision problem: corrections and explanations change the policy-side decision, generated rewards define the objective, and learned visual judges supply a training signal. Across all four, feedback phrasing and diversity are design variables rather than presentation details [92]. The modality changes the grounding burden, not the survey's inclusion rule. Extended real-robot, reward-model, and failure-recovery evidence appears in the electronic supplement.

11 Evaluation and Benchmarks

Evaluation must separate feedback *quality*, whether a signal is accurate, calibrated, and unbiased, from feedback *utilization*, whether the receiving system improves when that signal is supplied. The axes are independent: a correct critique can be ignored, while a reward model can score well on held-out preferences yet train a worse policy [36]. Treating end-task accuracy as a single summary obscures whether failure lies in the signal, the receiver, or their interface.

11.1 Measuring Feedback Quality

Quality benchmarks should match the signal's granularity and operational role. Outcome-preference benchmarks test reward-model ranking; RewardBench 2 is an anchor because it validates benchmark scores against best-of- N selection and PPO outcomes rather than assuming pairwise accuracy transfers [54]. Judge meta-evaluation tests verdict consistency and bias; JudgeBench ties labels to objective correctness and finds even strong judges only slightly above chance on hard pairs [78]. Critique benchmarks score error identification and correction; CriticBench evaluates generation, critique, and repair jointly [43]. Process benchmarks localize step-level errors; ProcessBench asks for the first erroneous reasoning step, directly testing the signal consumed by process supervision [118]. Embodied instruments test whether vision-language judges understand trajectories rather than final frames; ViSTa exposes severe weakness on sequential tasks [91]. These families are not interchangeable: a judge validated for reranking is not automatically a sound training reward or a useful critic.

Validity requires more than difficulty. Pairwise accuracy should predict the downstream use, and test distributions should expose length, style, refusal, and format. Process evaluation must separate error localization from final verdict because a wrong location misassigns credit. Generative critiques should separate factual correctness from actionable specificity. Report uncertainty and judge disagreement when small differences determine system selection.

11.2 Measuring Feedback Utilization

Utilization benchmarks score the receiver under a controlled signal. RealCritic measures a critique through the correction it induces and shows that plausible open-loop critiques can fail to improve self-revision [79]. Interactive environments isolate feedback uptake across attempts; LLF-Bench uses post-action language as the learning signal and paraphrases feedback to prevent task recognition from masquerading as learning [10]. Feedback Friction supplies near-oracle feedback yet finds that solvers still resist incorporating it, demonstrating that utilization can remain the bottleneck

after quality is nearly solved [32]. Long-horizon settings add retrieval and temporal credit: MemoryAgentBench tests whether accumulated verbal experience is retrieved, used, and selectively forgotten across episodes [29]. The detailed benchmark matrix and evidence catalog are provided in the electronic supplement.

A clean utilization study varies the signal while holding task, receiver, and budget fixed. A no-feedback control measures prior capability; an oracle condition estimates attainable gain; irrelevant feedback tests resistance to persuasive noise. Report improvement per interaction or token, and test several receivers because one critique can help a capable model yet overwhelm a weaker one.

11.3 Measurement Gaps and Minimum Reporting Protocol

Current evidence is strongest for short, verifiable, English, single-turn tasks and weakest for personalized, multilingual, long-horizon, adversarial, and embodied feedback. Judge-scored results inherit the judge’s consistency, bias, and aggregation uncertainty. Embodied benchmarks largely grade signal quality and rarely test whether a motor policy can use verbal feedback. Static evaluation also misses failures that emerge only when outputs change later observations or rewards.

These gaps limit claims of generality. Results from a short answer verifier do not establish trajectory-level credit, and an English critique benchmark does not establish that the same feedback interface transfers across languages or cultures. Evaluation should therefore state the intended deployment regime and treat movement beyond the benchmark’s signal type, horizon, modality, or interaction structure as an explicit extrapolation.

A minimum reporting protocol follows. (1) Report feedback quality and utilization separately. (2) Validate a quality benchmark against its downstream use, such as best-of- N selection, policy optimization, correction success, or task completion. (3) Audit judges and verifiers for consistency, bias, and adversarial robustness before using their verdicts as evaluation or reward. (4) State the signal type, granularity, interaction stage, and receiver, so results transfer only to mechanisms with the same role. When possible, include a no-feedback control and an oracle-feedback upper bound; their gap separates absent information from failed uptake. This protocol does not yet solve the lack of standardized utilization metrics, but it prevents quality and utilization failures from being conflated.

12 Safety and Robustness of Feedback Channels

Language is both supervision and an attack surface. Threats differ by who corrupts the signal and where it is consumed: an external adversary can manipulate a judge, observation, or training record; the policy can exploit its evaluator; and preference learning can inherit sycophancy without a deliberate attacker. The decomposition identifies which link of the verbal reward stack must be audited and why static capability evaluation is insufficient.

Downstream authority determines severity. A manipulated leaderboard changes a ranking; the same verdict used for learning changes a policy; an injected observation can trigger an external action. Medical, email, coding, and computer-use agents therefore need stricter controls because a linguistic error can become physical, financial, or privacy harm.

12.1 Robustness of LLM Judges

Injected phrases can force an LLM judge to prefer attacker-controlled outputs regardless of substance, corrupting evaluation, search, and training rewards. Optimized attacks outperform manual injections and known-answer or perplexity defenses [68]. Comparative scoring and judge commitments reduce exposure, but do not remove transferable artifacts or prompt-template sensitivity.

12.2 Injection into the Observation Loop

Tool outputs, webpages, and retrieved documents can carry indirect instructions that redirect an agent. InjecAgent demonstrates this attack against tool-integrated ReAct agents [107]. The defense boundary is therefore not the model prompt alone: untrusted observations require provenance, privilege separation, and constrained tool execution.

12.3 Poisoning of Training and Optimizer Feedback

Preference pairs and verbal optimizer feedback can be poisoned upstream. DPO backdoors succeed at lower poisoning rates than PPO-based pipelines in controlled comparisons [62], while language-space optimizers are especially exposed because a manipulated critique is their update direction. Data provenance and feedback-source trust must be reported with performance.

12.4 Policy Exploitation and Sycophancy

Even a clean channel can be exploited. Interactive feedback loops enable in-context reward hacking that static datasets cannot reveal [59]; underspecified rubrics similarly reward partial or superficial satisfaction. Sycophancy is the systemic counterpart: human and learned preference models can favor convincing agreement over correctness, steering optimization away from truthfulness [67]. Both require behavioral stress tests, not only held-out reward accuracy.

12.5 Defense Synthesis

Defenses fall into detection, signal reshaping, and system design. Detection alone is brittle because adaptive attacks optimize against the detector. Signal reshaping hardens what the reward credits, as adversarial verifier training does against master-key responses [117]. System design limits what corrupted text can cause through instruction hierarchy, provenance marking, least-privilege tools, and independent checks. Three rules survive across studies: comparative judgments are generally safer than absolute scores; stronger or multiple judges reduce but do not eliminate exploitation; and feedback-channel robustness must be measured under interaction, where attacks and reward hacking compound. A minimum safety report should name the trusted source, attack surface, downstream authority, stress tests, and residual failure mode. The electronic supplement catalogs the supporting attacks, quantitative results, and defenses.

Defense should follow the channel from source to effect. At ingestion, authenticate sources and preserve provenance instead of mixing trusted instructions with untrusted observations. At interpretation, compare independent judgments and test decisions under paraphrase and stylistic changes. At execution, gate high-impact tools, restrict permissions, and confirm irreversible actions. At learning time, inspect poisoning sensitivity and train rewards against high-scoring low-quality outputs. Evaluate the complete loop because reasonable filters, judges, and policies can still compose into an exploitable system. Report the adaptive attacker, utility cost, and authority available after compromise.

13 Practitioner Guide, Related Surveys, and Research Agenda

This section turns the taxonomy into a mechanism-selection rule, positions the survey against eighteen neighboring frameworks, and distills the open questions exposed by the synthesis.

13.1 Choosing a Verbal-Feedback Mechanism

Choose the feedback form before the optimizer. Four questions determine the mechanism in Figure 4. First, are model weights accessible? If so, use a compiled reward or rubric when outcomes are mechanically verifiable, monitoring objective hacking; otherwise, if verbal feedback is plentiful,

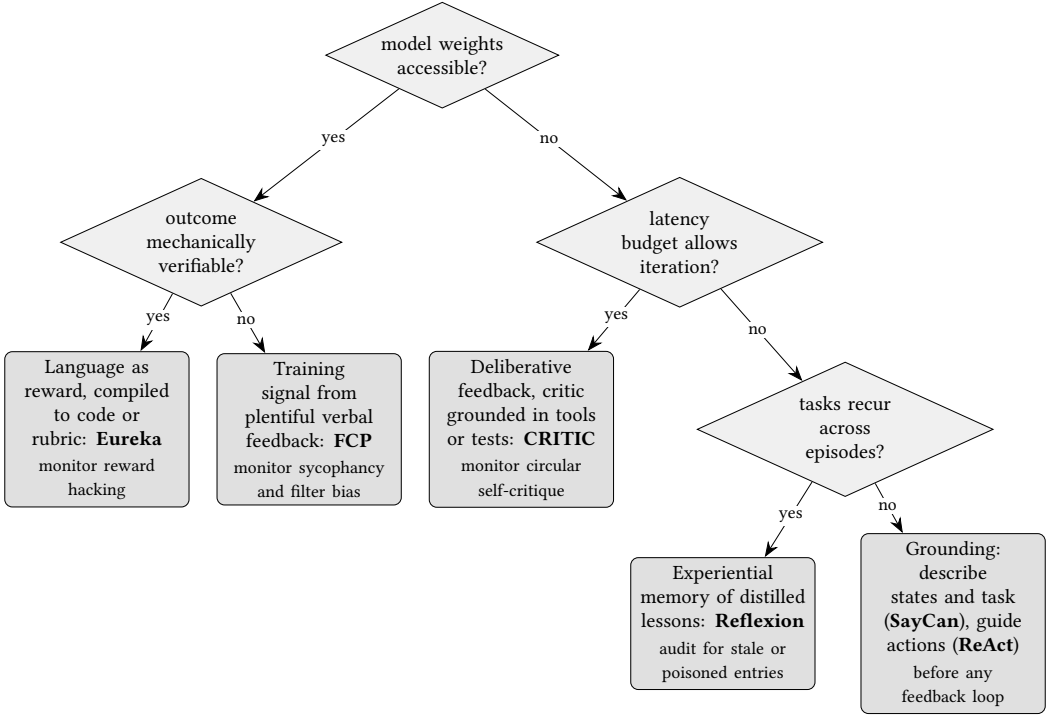


Fig. 4. Choosing a verbal-feedback mechanism through four ordered questions. Each leaf names a representative mechanism and its principal monitoring burden; the final leaf covers grounding before any feedback loop.

train on it directly and monitor sycophancy and filter bias [49]. Second, if weights are inaccessible but latency permits iteration, use deliberative feedback and ground the critic in tools or tests rather than relying on circular self-critique [24, 34]. Third, when iteration is too costly but tasks recur, persist distilled lessons and audit memory for stale or poisoned entries. Finally, if none of these preconditions holds, invest in grounding the task, state, or action interface before building a feedback loop. Each choice fixes a dominant risk as well as an optimization family.

13.2 Comparison with Prior Surveys

To our knowledge, this survey is the first organized around the verbal signal itself: which component of the decision problem a linguistic artifact modifies and when it takes effect. Table 4 makes the positioning explicit against eighteen prior surveys and frameworks. Most shelf neighbors organize by mechanism (correction timing, evolution phase, rollout stage), by agent component (memory, capability, module under optimization), or by algorithm family (RLHF, preference optimization, agentic RL); the nearest exception, the feedback typing of Fernandes et al. [22], does take the signal as its axis but predates the agentic signals surveyed here. None combines a signal-centric axis with a formal framework, coverage of all three stages (problem definition, inference, and training), and treatment of the feedback channel's safety; that fourfold combination is the position this survey occupies.

Table 4. Positioning against prior surveys and frameworks. Feedback types: S = scalar, P = preference, V = verbal; a letter in parentheses marks a feedback type the work treats only partially. Stages: D = problem definition and grounding, I = inference, T = training. Formal (formal framework), Guide (prescriptive practitioner guidance), and Safety (feedback-channel safety): Y = yes, P = partial, N = not claimed. All cells are graded from each work’s abstract and stated contributions, so N records the absence of a claim there, not a judgment of the work’s full text.

Survey	Scope	Organizing axis	Feedb.	Stages	Formal	Guide	Safety
Luketina et al. [48]	language-informed RL (pre-LLM)	task setting	S	T	N	N	N
Pan et al. [60]	LLM self-correction	correction timing	V, (S)	I, T	N	P	N
Kamoi et al. [34]	self-correction critique	research question $\times$ feedback source	V	I, T	N	Y	N
Fernandes et al. [22]	feedback for generation	format $\times$ objective $\times$ use	S, P, V	I, T	Y	P	P
Wang et al. [87]	RL-enhanced LLMs	training technique (RLHF, RLAI, DPO)	S, P	T	N	P	N
Xiao et al. [94]	preference optimization	research questions	P	T	P	P	N
Zhang et al. [114]	agent memory	design, evaluation, application	(V)	I	P	P	N
Zhang et al. [113]	test-time scaling	what/how/where/how well	(S), (V)	I	P	Y	N
Gao et al. [23]	self-evolving agents	what/when/how to evolve	S, V	I, T	P	P	Y
Fang et al. [20]	self-evolving agents	component under optimization	(S), (V)	I, T	Y	P	Y
Du et al. [17]	LLM-agent optimization	parameter-driven vs. parameter-free	S, (V)	I, T	N	P	N
Zhang et al. [109]	agentic RL	capability $\times$ domain	S, (P)	T	Y	P	N
Srivastava and Aggarwal [74]	RL for LLMs	reward $\times$ feedback $\times$ optimization	S, P, (V)	T	Y	Y	P
Zhang [108]	LLM-RL credit assignment	granularity $\times$ methodology	S	T	N	Y	N
Ji et al. [31]	alignment	reward-mechanism dimensions	S, P	T	Y	Y	N
Wu [90]	learning from rewards	reward model $\times$ strategy $\times$ stage	S, P, (V)	I, T	N	P	N
Surana et al. [75]	RL rollout design	rollout stage	S, (V)	T	Y	Y	P
Feng et al. [21]	NLRL (framework)	RL-component analogues in language	V	I, T	Y	N	N
This survey	verbal RL	the signal: what it modifies, when it acts	V, (S), (P)	D, I, T	Y	Y	Y

13.3 Research Agenda

The synthesis exposes ten linked gaps. Theory must identify what language contributes beyond scalarization, while systems work must make feedback trustworthy and usable.

- (1) Characterize when verbal feedback is more sample efficient than scalar reward and extend current separations to intermediate formats [98].
- (2) Establish convergence conditions for compiled and rubric-shaped rewards under objective mis-specification.
- (3) Assign free-form critiques to the responsible spans, steps, and decisions rather than only to outcomes [84].
- (4) Measure information lost when a critique is compressed to a preference label or score.
- (5) Authenticate feedback provenance and evaluate adaptive injection, memory poisoning, and reward manipulation [106].
- (6) Report feedback quality separately from the receiving model's ability to use accurate feedback [32].
- (7) Train calibrated critics for error localization and actionable correction instead of reusing the generator uncritically [55].
- (8) Standardize machine-readable tool feedback that separates task errors from infrastructure failures.
- (9) Formalize when persistent memory or replay crosses the boundary from inference to training.
- (10) Test transfer, validity, and affected-party harms in embodied and high-stakes deployments [67].

14 Conclusion

This survey organizes verbal reinforcement learning by where a natural-language signal enters a decision problem and what it modifies when it takes effect. The resulting pillars separate grounding at problem-definition time, deliberative feedback during inference, and learning signals that persist through training. A two-criterion fence excludes plain instruction following and feedback scalarized before use, while the assignment rule places hybrid methods by each signal's point of effect. This signal-centric view connects topics often treated separately: reward construction, critique, memory, credit assignment, language-space optimization, evaluation, and feedback-channel safety.

The practical implication is that feedback format cannot be selected independently of its consumer. A verifier, free-form critique, stored lesson, and training rationale carry different information, require different credit mechanisms, and expose different attack surfaces. The field therefore needs theory that measures what survives scalarization, evaluations that separate feedback quality from utilization, and provenance-aware interfaces that make external feedback auditable. These priorities determine whether language becomes a dependable control signal rather than another unverified model output.

14.1 Limitations

This article synthesizes representative work rather than exhaustively enumerating every system. Its taxonomy is an organizing lens, not a strict partition, because one system may consume several signal roles. Scope is restricted to explicit textual feedback; spoken and richer multimodal feedback fall outside the fence even when observations are visual. The survey contributes no new benchmark or controlled comparison, and formal guarantees remain localized. Evidence for real-world transfer is still sparse, especially in embodied and high-stakes settings, so successful demonstrations do not establish broad reliability. Where the literature supplies no guarantee or comparative evidence, we state the gap rather than infer a conclusion.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, and Shyamal Anadkat. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, and Meng Jiang. 2025. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457* (2025).
- [3] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, and Karol Hausman. 2022. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691* (2022).
- [4] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565* (2016).
- [5] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. 2022. Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073* (2022).
- [6] Vineet Bhat, Ali Umut Kaypak, Prashanth Krishnamurthy, Ramesh Karri, and Farshad Khorrami. 2024. Grounding llms for robot task planning using closed-loop state feedback. *arXiv preprint arXiv:2402.08546* (2024).
- [7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, and Amanda Askell. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [8] Yekun Chai, Haoran Sun, Huang Fang, Shuohuan Wang, Yu Sun, and Hua Wu. 2024. MA-RLHF: Reinforcement Learning from Human Feedback with Macro Actions. (2024). arXiv:2410.02743
- [9] Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. 2023. ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate. arXiv:2308.07201 [cs.CL] <https://arxiv.org/abs/2308.07201>
- [10] Ching-An Cheng, Andrey Kolobov, Dipendra Misra, Allen Nie, and Adith Swaminathan. 2023. LLF-Bench: Benchmark for Interactive Learning from Language Feedback. (2023). arXiv:2312.06853
- [11] Ching-An Cheng, Allen Nie, and Adith Swaminathan. 2024. Trace is the Next AutoDiff: Generative Optimization with Rich Feedback, Execution Traces, and LLMs. (2024). arXiv:2406.16218
- [12] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems* 30 (2017). arXiv:1706.03741
- [13] Yuchen Cui, Siddharth Karamcheti, Raj Pallethi, Nidhya Shivakumar, Percy Liang, and Dorsa Sadigh. 2023. “No, to the Right” – Online Language Corrections for Robotic Manipulation via Shared Autonomy. (2023). arXiv:2301.02555
- [14] Parth Darshan and Abhishek Divekar. 2026. When Gradients Collide: Failure Modes of Multi-Objective Prompt Optimization for LLM Judges. (2026). arXiv:2605.26046
- [15] Sarkar Snigdha Sarathi Das, Ryo Kamoi, Bo Pang, Yusen Zhang, Caiming Xiong, and Rui Zhang. 2024. GReaTer: Gradients over Reasoning Makes Smaller Language Models Strong Prompt Optimizers. (2024). arXiv:2412.09722
- [16] Zhichen Dong, Yang Li, Yuhuan Sun, Weixun Wang, Yijia Luo, Zinian Peng, Taiheng Ye, Chao Yang, Wenbo Su, Yu Cheng, Bo Zheng, and Junchi Yan. 2026. How Does Reasoning Flow? Tracing Attention-Induced Information Flow for Targeted RL in LLMs. (2026). arXiv:2606.10646
- [17] Shangheng Du, Jiabao Zhao, Jinxin Shi, Zhentao Xie, Xin Jiang, Yanhong Bai, and Liang He. 2026. A Survey on the Optimization of Large Language Model-based Agents. *Comput. Surveys* 58, 9 (2026). arXiv:2503.12434
- [18] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2024. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In *International Conference on Machine Learning*.
- [19] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. 2024. AHA: A Vision-Language-Model for Detecting and Reasoning Over Failures in Robotic Manipulation. (2024). arXiv:2410.00371
- [20] Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. 2025. A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems. (2025). arXiv:2508.07407
- [21] Xidong Feng, Bo Liu, Yan Song, Haotian Fu, Ziyu Wan, Girish A. Koushik, Zhiyuan Hu, Mengyue Yang, Ying Wen, and Jun Wang. 2024. Natural Language Reinforcement Learning. (2024). arXiv:2411.14251

- [22] Patrick Fernandes, Aman Madaan, Emmy Liu, António Farinhas, Pedro Henrique Martins, Amanda Bertsch, José G. C. de Souza, Shuyan Zhou, Tongshuang Wu, Graham Neubig, and André F. T. Martins. 2023. Bridging the Gap: A Survey on Integrating (Human) Feedback for Natural Language Generation. (2023). [arXiv:2305.00955](#)
- [23] Huan-ang Gao, Jiayi Geng, Wenye Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Qihan Ren, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. 2025. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. (2025). [arXiv:2507.21046](#)
- [24] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*. [arXiv:2305.11738](#)
- [25] Anisha Gunjal, Anthony Wang, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx. 2025. Rubrics as Rewards: Reinforcement Learning Beyond Verifiable Domains. (2025). [arXiv:2507.17746](#)
- [26] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, and Xiao Bi. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [27] Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 8154–8173.
- [28] Jianliang He, Siyu Chen, Fengzhuo Zhang, and Zhuoran Yang. 2024. From Words to Actions: Unveiling the Theoretical Underpinnings of LLM-Driven Autonomous Systems. (2024). [arXiv:2405.19883](#)
- [29] Yuanzhe Hu, Yu Wang, and Julian McAuley. 2025. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. (2025). [arXiv:2507.05257](#)
- [30] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, and Yevgen Chebotar. 2022. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608* (2022).
- [31] Miaomiao Ji, Yanqiu Wu, Zhibin Wu, Shoujin Wang, Jian Yang, Mark Dras, and Usman Naseem. 2025. A Survey on Progress in LLM Alignment from the Perspective of Reward Design. (2025). [arXiv:2505.02666](#)
- [32] Dongwei Jiang, Alvin Zhang, Andrew Wang, Nicholas Andrews, and Daniel Khoshnab. 2025. Feedback Friction: LLMs Struggle to Fully Incorporate External Feedback. (2025). [arXiv:2506.11930](#)
- [33] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. 1998. Planning and acting in partially observable stochastic domains. *Artificial Intelligence* 101, 1–2 (1998), 99–134. [doi:10.1016/S0004-3702\(98\)00023-X](#)
- [34] Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. 2024. When can llms actually correct their own mistakes? a critical survey of self-correction of llms. *Transactions of the Association for Computational Linguistics* 12 (2024), 1417–1440.
- [35] Muhammad Khalifa, Rishabh Agarwal, Lajanugen Logeswaran, Jaekyeom Kim, Hao Peng, Moontae Lee, Honglak Lee, and Lu Wang. 2025. Process Reward Models That Think. [arXiv:2504.16828 \[cs.LG\]](#) <https://arxiv.org/abs/2504.16828>
- [36] Sunghwan Kim, Dongjin Kang, Taeyoon Kwon, Hyungjoo Chae, Jungsoo Won, Dongha Lee, and Jinyoung Yeo. 2024. Evaluating Robustness of Reward Models for Mathematical Reasoning. (2024). [arXiv:2410.01729](#)
- [37] Ryan Koo, Ian Yang, Vipul Raheja, Mingyi Hong, Kwang-Sung Jun, and Dongyeop Kang. 2025. Learning Explainable Dense Reward Shapes via Bayesian Optimization. [arXiv:2504.16272](#)
- [38] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, and Shane Lyu. 2024. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124* (2024).
- [39] Andrew Li, Torny Klassen, Andrew Wang, Parand A Alamdari, and Sheila McIlraith. 2026. Ground-Compose-Reinforce: Grounding Language in Agentic Behaviours using Limited Data. *Advances in Neural Information Processing Systems* 38 (2026), 160838–160874.
- [40] Jiahui Li, Lin Li, Tai-wei Chang, Kun Kuang, Long Chen, Jun Zhou, and Cheng Yang. 2024. RED: Unleashing Token-Level Rewards from Holistic Feedback via Reward Redistribution. (2024). [arXiv:2411.08302](#)
- [41] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let’s verify step by step. In *International Conference on Learning Representations*, Vol. 2024. 39578–39601. [arXiv:2305.20050](#)
- [42] Muhan Lin, Shuyang Shi, Yue Guo, Vaishnav Tadiparthi, Behdad Chalaki, Ehsan Moradi Pari, Simon Stepputtis, Woojun Kim, Joseph Campbell, and Katia Sycara. 2025. Speaking the Language of Teamwork: LLM-Guided Credit Assignment in Multi-Agent Reinforcement Learning. (2025). [arXiv:2502.03723](#)
- [43] Zicheng Lin, Zhibin Gou, Tian Liang, Ruilin Luo, Haowei Liu, and Yujiu Yang. 2024. Criticbench: Benchmarking llms for critique-correct reasoning. In *Findings of the Association for Computational Linguistics: ACL 2024*. 1552–1587.

- [44] Xiaoqian Liu, Ke Wang, Yuchuan Wu, Fei Huang, Yongbin Li, Junge Zhang, and Jianbin Jiao. 2025. Agentic Reinforcement Learning with Implicit Step Rewards. *arXiv:2509.19199* [cs.CL] <https://arxiv.org/abs/2509.19199>
- [45] Zijun Liu, Peiyi Wang, Runxin Xu, Shirong Ma, Chong Ruan, Peng Li, Yang Liu, and Yu Wu. 2025. Inference-Time Scaling for Generalist Reward Modeling. (2025). *arXiv:2504.02495*
- [46] Saüc Abadal Lloret, Shehzaad Dhuliawala, Keerthiram Murugesan, and Mrinmaya Sachan. 2024. Towards aligning language models with textual feedback. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 20240–20266.
- [47] Xinyu Lu, Kaiqi Zhang, Jinglin Yang, Boxi Cao, Yaojie Lu, Hongyu Lin, Min He, Xianpei Han, and Le Sun. 2026. P²O: Joint Policy and Prompt Optimization. (2026). *arXiv:2603.21877*
- [48] Jelena Luketina, Nantas Nardelli, Gregory Farquhar, Jakob Foerster, Jacob Andreas, Edward Grefenstette, Shimon Whiteson, and Tim Rocktäschel. 2019. A survey of reinforcement learning informed by natural language. *arXiv preprint arXiv:1906.03926* (2019).
- [49] Renjie Luo, Zichen Liu, Xiangyan Liu, Chao Du, Min Lin, Wenhu Chen, Wei Lu, and Tianyu Pang. 2025. Language models can learn from verbal feedback without scalar rewards. *arXiv preprint arXiv:2509.22638* (2025).
- [50] Ruotian Ma, Xiaolei Wang, Xin Zhou, Jian Li, Nan Du, Tao Gui, Qi Zhang, and Xuanjing Huang. 2024. Are Large Language Models Good Prompt Optimizers? (2024). *arXiv:2402.02101*
- [51] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, and Jim Fan. 2024. Eureka: Human-level reward design via coding large language models. In *International conference on learning Representations*, Vol. 2024. 26516–26560.
- [52] James MacGlashan, Mark K Ho, Robert Loftin, Bei Peng, Guan Wang, David L Roberts, Matthew E Taylor, and Michael L Littman. 2017. Interactive learning from policy-dependent human feedback. In *International conference on machine learning*. PMLR, 2285–2294. *arXiv:1701.06049*
- [53] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*.
- [54] Saumya Malik, Valentina Pyatkin, Sander Land, Jacob Morrison, Noah A. Smith, Hannaneh Hajishirzi, and Nathan Lambert. 2025. RewardBench 2: Advancing Reward Model Evaluation. (2025). *arXiv:2506.01937*
- [55] Nat McAleese, Rai Michael Pokorný, Juan Felipe Ceron Uribe, Evgenia Nitishinskaya, Maja Trebacz, and Jan Leike. 2024. LLM Critics Help Catch LLM Bugs. *arXiv preprint arXiv:2407.00215* (2024).
- [56] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, and Georg Ostrovski. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533.
- [57] Sumeet Ramesh Motwani, Chandler Smith, Rocktim Jyoti Das, Rafael Rafailov, Ivan Laptev, Philip H. S. Torr, Fabio Pizzati, Ronald Clark, and Christian Schroeder de Witt. 2024. MALT: Improving Reasoning with Multi-Agent LLM Training. (2024). *arXiv:2412.01928*
- [58] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, and Alex Ray. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [59] Alexander Pan, Erik Jones, Meena Jagadeesan, and Jacob Steinhardt. 2024. Feedback Loops With Language Models Drive In-Context Reward Hacking. *arXiv:2402.06627*
- [60] Liangming Pan, Michael Saxon, Wenda Xu, Deepak Nathani, Xinyi Wang, and William Yang Wang. 2023. Automatically Correcting Large Language Models: Surveying the landscape of diverse self-correction strategies. (2023). *arXiv:2308.03188*
- [61] Chanwoo Park, Seungju Han, Xingzhi Guo, Asuman Ozdaglar, Kaiqing Zhang, and Joo-Kyung Kim. 2025. MAPoRL: Multi-Agent Post-Co-Training for Collaborative Large Language Models with Reinforcement Learning. (2025). *arXiv:2502.18439*
- [62] Pankayaraj Pathmanathan, Souradip Chakraborty, Xiangyu Liu, Yongyuan Liang, and Furong Huang. 2024. Is poisoning a real threat to LLM alignment? Maybe more so than you think. *arXiv:2406.12091*
- [63] Eduardo Pignatelli, Johan Ferret, Tim Rocktäschel, Edward Grefenstette, Davide Paglieri, Samuel Coward, and Laura Toni. 2024. Assessing the Zero-Shot Capabilities of LLMs for Action Evaluation in RL. (2024). *arXiv:2409.12798*
- [64] Yun Qu, Yuhang Jiang, Boyuan Wang, Yixiu Mao, Cheems Wang, Chang Liu, and Xiangyang Ji. 2024. Latent Reward: LLM-Empowered Credit Assignment in Episodic Reinforcement Learning. (2024). *arXiv:2412.11120*
- [65] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741.

- [66] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [67] Mrinank Sharma, Meg Tong, Tomek Korbak, David Duvenaud, Amanda Asbell, Sam Bowman, Esin Durmus, Zac Hatfield-Dodds, Scott Johnston, and Shauna Kravec. 2024. Towards understanding sycophancy in language models. In *International Conference on Learning Representations*, Vol. 2024. 110–144.
- [68] Jiawen Shi, Zenghui Yuan, Yinuo Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. 2024. Optimization-based Prompt Injection Attack to LLM-as-a-Judge. *arXiv:2403.17710*
- [69] Noah Shinn, Federico Cassano, Berman Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [70] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2020. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768* (2020).
- [71] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, and Marc Lanctot. 2016. Mastering the game of Go with deep neural networks and tree search. *nature* 529, 7587 (2016), 484–489.
- [72] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2022. Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302* (2022).
- [73] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters. *arXiv preprint arXiv:2408.03314* (2024).
- [74] Saksham Sahai Srivastava and Vaneet Aggarwal. 2025. A Technical Survey of Reinforcement Learning Techniques for Large Language Models. (2025). *arXiv:2507.04136*
- [75] Rohan Surana, Gagan Mundada, Xunyi Jiang, Chuhan Wang, Zhenwei Tang, Difan Jiao, Zihan Huang, Yuxin Xiong, Junda Wu, Sheldon Yu, Xintong Li, Raghav Jain, Nikki Kuang, Sizhe Zhou, Bowen Jin, Zhendong Chu, Tong Yu, Ryan Rossi, Kuan-Hao Huang, Jingbo Shang, Jiawei Han, and Julian McAuley. 2026. Generate, Filter, Control, Replay: A Comprehensive Survey of Rollout Strategies for LLM Reinforcement Learning. (2026). *arXiv:2605.02913*
- [76] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement Learning: An Introduction* (2 ed.). MIT Press.
- [77] Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2026. Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vera Demberg, Kentaro Inui, and Lluís Marquez (Eds.). Association for Computational Linguistics, Rabat, Morocco, 7080–7106. doi:10.18653/v1/2026.eacl-long.333
- [78] Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. 2024. JudgeBench: A Benchmark for Evaluating LLM-based Judges. (2024). *arXiv:2410.12784*
- [79] Zhengyang Tang, Ziniu Li, Zhenyang Xiao, Tian Ding, Ruoyu Sun, Benyou Wang, Dayiheng Liu, Fei Huang, Tianyu Liu, Bowen Yu, and Junyang Lin. 2025. RealCritic: Towards Effectiveness-Driven Evaluation of Language Model Critiques. (2025). *arXiv:2501.14492*
- [80] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275* (2022).
- [81] Peter Vamplew, Benjamin J. Smith, Johan Kallstrom, Gabriel Ramos, Roxana Radulescu, Diederik M. Roijers, Conor F. Hayes, Fredrik Heintz, Patrick Mannion, Pieter J. K. Libin, Richard Dazeley, and Cameron Foale. 2021. Scalar reward is not enough: A response to Silver, Singh, Precup and Sutton (2021). *arXiv:2112.15422 [cs.AI]* <https://arxiv.org/abs/2112.15422>
- [82] Vijay Viswanathan, Yanchao Sun, Shuang Ma, Xiang Kong, Meng Cao, Graham Neubig, and Tongshuang Wu. 2025. Checklists Are Better Than Reward Models For Aligning Language Models. (2025). *arXiv:2507.18624*
- [83] Guoqing Wang, Sunhao Dai, Guangze Ye, Zeyu Gan, Wei Yao, Yong Deng, Xiaofeng Wu, and Zhenzhe Ying. 2025. Information Gain-based Policy Optimization: A Simple and Effective Approach for Multi-Turn Search Agents. (2025). *arXiv:2510.14967*
- [84] Hanyang Wang, Lu Wang, Chaoyun Zhang, Tianjun Mao, Si Qin, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025. Text2Grad: Reinforcement Learning from Natural Language Feedback. *arXiv preprint arXiv:2505.22338* (2025).
- [85] Jianyu Wang, Zhiqiang Hu, and Lidong Bing. 2025. Evolving Prompts In-Context: An Open-ended, Self-replicating Perspective. (2025). *arXiv:2506.17930*
- [86] Peiyi Wang, Lei Li, Zhihong Shao, RX Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024. Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations. *arXiv preprint arXiv:2312.08935* (2024).

- [87] Shuhe Wang, Shengyu Zhang, Jie Zhang, Runyi Hu, Xiaoya Li, Tianwei Zhang, Jiwei Li, Fei Wu, Guoyin Wang, and Eduard Hovy. 2024. Reinforcement Learning Enhanced LLMs: A Survey. (2024). [arXiv:2412.10400](#)
- [88] Yufei Wang, Zhanyi Sun, Jesse Zhang, Zhou Xian, Erdem Biyik, David Held, and Zackory Erickson. 2024. RL-VLM-F: Reinforcement Learning from Vision Language Foundation Model Feedback. (2024). [arXiv:2402.03681](#)
- [89] Quan Wei, Siliang Zeng, Chenliang Li, William Brown, Oana Frunza, Wei Deng, Anderson Schneider, Yuriy Nevmyvaka, Yang Katie Zhao, Alfredo Garcia, and Mingyi Hong. 2025. Reinforcing Multi-Turn Reasoning in LLM Agents via Turn-Level Reward Design. (2025). [arXiv:2505.11821](#)
- [90] Xiaobao Wu. 2025. Sailing by the Stars: A Survey on Reward Models and Learning Strategies for Learning from Rewards. (2025). [arXiv:2505.02686](#)
- [91] Evžen Wybitul, Evan Ryan Gunter, Mikhail Seleznyov, and David Lindner. 2024. ViSta Dataset: Do vision-language models understand sequential tasks? (2024). [arXiv:2411.13211](#)
- [92] Jiajun Xi, Yinong He, Jianing Yang, Yinpei Dai, and Joyce Chai. 2024. Teaching Embodied Reinforcement Learning Agents: Informativeness and Diversity of Language Use. (2024). [arXiv:2410.24218](#)
- [93] Tim Z. Xiao, Robert Bamler, Bernhard Schölkopf, and Weiyang Liu. 2024. Verbalized Machine Learning: Revisiting Machine Learning with Language Models. (2024). [arXiv:2406.04344](#)
- [94] Wenyi Xiao, Zechuan Wang, Leilei Gan, Shuai Zhao, Zongrui Li, Ruirui Lei, Wanggui He, Luu Anh Tuan, Long Chen, Hao Jiang, Zhou Zhao, and Fei Wu. 2024. A Comprehensive Survey of Direct Preference Optimization: Datasets, Theories, Variants, and Applications. (2024). [arXiv:2410.15595](#)
- [95] Guofu Xie, Yunsheng Shi, Hongtao Tian, Ting Yao, and Xiao Zhang. 2025. CAPO: Towards Enhancing LLM Reasoning through Generative Credit Assignment. (2025). [arXiv:2508.02298](#)
- [96] Tianbao Xie, Siheng Zhao, Chen Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. 2024. Text2reward: Reward shaping with language models for reinforcement learning. In *International Conference on Learning Representations*, Vol. 2024. 35663–35699.
- [97] Zhihui Xie, Jie Chen, Liyu Chen, Weichao Mao, Jingjing Xu, and Lingpeng Kong. 2025. Teaching Language Models to Critique via Reinforcement Learning. (2025). [arXiv:2502.03492](#)
- [98] Wanqiao Xu, Allen Nie, Ruijie Zheng, Aditya Modi, Adith Swaminathan, and Ching-An Cheng. 2025. Formalizing Learning from Language Feedback with Provable Guarantees. (2025). [arXiv:2506.10341](#)
- [99] Sikuan Yan, Ahmed Bahloul, Ercong Nie, Susanna Schwarzmann, Riccardo Trivisonno, Volker Tresp, and Yunpu Ma. 2026. Memory-R2: Fair Credit Assignment for Long-Horizon Memory-Augmented LLM Agents. (2026). [arXiv:2605.21768](#)
- [100] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. In *International Conference on Learning Representations Workshop*. [arXiv:2309.03409](#)
- [101] Wenkai Yang, Jingwen Chen, Yankai Lin, and Ji-Rong Wen. 2025. DeepCritic: Deliberate Critique with Large Language Models. (2025). [arXiv:2505.00662](#)
- [102] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
- [103] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. 2024. Self-rewarding language models. *arXiv preprint arXiv:2401.10020* (2024).
- [104] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. TextGrad: Automatic “Differentiation” via Text. *arXiv preprint arXiv:2406.07496* (2024).
- [105] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D Goodman. 2022. STaR: Bootstrapping Reasoning With Reasoning. In *Advances in Neural Information Processing Systems*.
- [106] Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. 2025. Adaptive attacks break defenses against indirect prompt injection attacks on llm agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 7101–7117.
- [107] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506.
- [108] Chenchen Zhang. 2026. From Reasoning to Agentic: Credit Assignment in Reinforcement Learning for Large Language Models. (2026). [arXiv:2604.09459](#)
- [109] Guibin Zhang, Hejia Geng, Xiaohang Yu, Zhenfei Yin, Zaibin Zhang, Zelin Tan, Heng Zhou, Zhongzhi Li, Xiangyuan Xue, Yijiang Li, Yifan Zhou, Yang Chen, Chen Zhang, Yutao Fan, Zihu Wang, Songtao Huang, Francisco Piedrahita-Velez, Yue Liao, Hongru Wang, Mengyue Yang, Heng Ji, Jun Wang, Shuicheng Yan, Philip Torr, and Lei Bai. 2025. The Landscape of Agentic Reinforcement Learning for LLMs: A Survey. (2025). [arXiv:2509.02547](#)
- [110] Hangfan Zhang, Zhiyao Cui, Jianhao Chen, Xinrun Wang, Qiaosheng Zhang, Zhen Wang, Dinghao Wu, and Shuyue Hu. 2025. Stop Overvaluing Multi-Agent Debate – We Must Rethink Evaluation and Embrace Model Heterogeneity.

- (2025). [arXiv:2502.08788](#)
- [111] Lunjun Zhang, Arian Hosseini, Hritik Bansal, Mehran Kazemi, Aviral Kumar, and Rishabh Agarwal. 2025. Generative Verifiers: Reward Modeling as Next-Token Prediction. In *International Conference on Learning Representations*. [arXiv:2408.15240](#)
 - [112] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2025. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. (2025). [arXiv:2510.04618](#)
 - [113] Qiyuan Zhang, Fuyuan Lyu, Zexu Sun, Lei Wang, Weixu Zhang, Wenyue Hua, Haolun Wu, Zhihan Guo, Yufei Wang, Niklas Muennighoff, Irwin King, Xue Liu, and Chen Ma. 2025. A Survey on Test-Time Scaling in Large Language Models: What, How, Where, and How Well? (2025). [arXiv:2503.24235](#)
 - [114] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems* 43, 6 (2025), 1–47.
 - [115] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. In *AAAI Conference on Artificial Intelligence*.
 - [116] Jian Zhao, Runze Liu, Kaiyan Zhang, Zhimu Zhou, Junqi Gao, Dong Li, Jiafei Lyu, Zhouyi Qian, Bqing Qi, and Xiu Li. 2026. Genprm: Scaling test-time compute of process reward models via generative reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 34932–34940.
 - [117] Yulai Zhao, Haolin Liu, Dian Yu, Sunyuan Kung, Meijia Chen, Haitao Mi, and Dong Yu. 2025. One Token to Fool LLM-as-a-Judge. [arXiv:2507.08794](#)
 - [118] Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2024. ProcessBench: Identifying Process Errors in Mathematical Reasoning. (2024). [arXiv:2412.06559](#)
 - [119] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, and Eric Xing. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
 - [120] Victor Zhong, Dipendra Misra, Xingdi Yuan, and Marc-Alexandre Côté. 2024. Policy Improvement using Language Feedback Models. (2024). [arXiv:2402.07876](#)
 - [121] Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, and Jun Wang. 2025. Memento: Fine-tuning LLM Agents without Fine-tuning LLMs. (2025). [arXiv:2508.16153](#)
 - [122] Wenxuan Zhou, Shujian Zhang, Lingxiao Zhao, and Tao Meng. 2024. T-REG: Preference Optimization with Token-Level Reward Regularization. (2024). [arXiv:2412.02685](#)
 - [123] Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. 2024. ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL. (2024). [arXiv:2402.19446](#)